Orkestrasyon MCP incelemesi

16 Ekim 2026 | Yerel kod incelemesi ve izole testler

Görüşüm

MCP bağlantısı gerçek ve çalışıyor. SQLite üzerinde görev sırası, bağımlılık ve oturum durumu tutan kullanılabilir bir prototip var. Buna karşılık mevcut uygulamayı güvenilir bir çoklu işçi orkestratörü olarak kullanıma hazır bulmuyorum. Başarı kapısı ajanın beyanına güveniyor; dosya yazma sınırı aşılabiliyor; görev sahipliği ve paralel dağıtım uygulanmıyor.

Bu sürüm eski filoyu MCP'ye bağlamamış. Sunucu işçi veya model başlatmıyor, havuzdan hesap almıyor, worktree oluşturmuyor ve kabul komutunu kendisi çalıştırmıyor. Scout -> Builder -> Reviewer -> Compactor sırasını ana ajana rehber metniyle bildiriyor. Rol değiştirmek kendi başına ayrı ajan veya ayrı bağlam oluşturmuyor. Bu, 4 kaynak dosyanın incelenmesine dayanan mimari değerlendirmedir.

Bulunan kurulum

• Kaynak: C:/Users/Enes/.gemini/antigravity/mcp_servers/orchestrator/

• Dosyalar: server.py, dag.py, db.py, test_orchestrator.py.

• Antigravity bağlantısı: C:/Users/Enes/.gemini/config/mcp_config.json, orchestrator girdisi.

• 8 araç: init, add_task, get_next_action, record_result, save_artifact, get_status, list_sessions, reset_session (hepsinde orchestrator_ öneki var).

• Testte kurulan gerçek stdio bağlantısı MCP 2025-06-18 protokolünü ve antigravity-orchestrator sunucu kimliğini döndürdü. Sunucunun ve kurulu Python paketinin sürüm etiketi 2.0.0.

Gerçek test sonucu

Küme	Sonuç	Anlamı
Mevcut birim testleri	6/6 geçti, ayrı komut rc=0	Normal akış, döngü denetimi, hata yayılımı, temel sıfırlama, listeleme, SQLite WAL
Ek stdio testleri	6/6 geçti	Handshake, 8 araç, komut sonucunu taşıma, eksik argümanı reddetme, rc=7 sonrası durdurma, süreç yeniden başlayınca devam
Bağımsız sınır testleri	0/10 geçti	Aşağıdaki dayanıklılık ve güven sorunlarını ortaya çıkardı
Toplam	22 test; 12 geçti, 10 başarısız, 0 test hatası; rc=1	Bu sayı genel kalite puanı değildir

Stdio testlerinden biri kusurları karakterize eder: kusurlu davranışın gerçek MCP üzerinden de tekrarlandığını doğruladığı için geçer. Dolayısıyla 12 geçen testin tamamı olumlu ürün davranışı anlamına gelmez. Mevcut testler fonksiyonları doğrudan çağırır; ek testlerde gerçekten yeni sunucu süreçleri açılıp JSON-RPC mesajları gönderildi.

Öncelikli bulgular

1. **P1 - Kabul kapısı bağımsız doğrulama yapmıyor.** Hiç komut koşmadan, olmayan kanıt dosyası vererek SUCCESS bildirdim; is_gate_passed=true döndü. exit_code atlanırsa varsayılanı 0. Sunucu gerçek süreç sonucunu üretmiyor veya kanıt dosyasının varlığını bile denetlemiyor. Testlerin geçtiğini söyleyen ajanın yanılması bu kapıdan geçer. Kaynak: server.py:161-216.

2. **P1 - Dosya yazımı çıktı klasöründen kaçabiliyor.** session_id='../wire-escaped' ile sunucu belirlenen artifact kökünün dışına dosya yazdı. Yalnız testin geçici klasöründe yeni dosya oluşturuldu. Oturum/görev varlığı ve çözülmüş hedef yolun kök içinde kaldığı kontrol edilmiyor. Sunucunun erişebildiği dosyalara kapsam dışı yazma riski var. Kaynak: server.py:221-240.

3. **P1 - Görev geçişleri doğrulanmıyor.** Scout ve Builder bitmeden bekleyen Reviewer'ı COMPLETED yaptım. Var olmayan görev için de başarılı kabul yanıtı döndü; iteration arttı. Bu iki davranış gerçek MCP çağrısıyla da tekrarlandı. Kaynak: server.py:184-216, db.py:265-292.

Diğer doğrulanan sınırlar

Öncelik	Deney	Gözlenen davranış	Kaynak
P2	max_iterations=1 sonrası devam	Sınır aşıldıktan sonra yeni görev veriliyor; MCP üzerinden de tekrarlandı	dag.py:107-191
P2 / kapasite	Bağımsız A ve B için iki görev isteği	İkisi de A'yı alıyor; ikinci yanıt resume sayılıyor. Tek istemci için devam özelliği, çoklu işçi dağıtımı için yetersiz	dag.py:134-145
P2	Aynı sonucu iki kez gönderme	Aynı iş için iteration 1 yerine 2 oluyor; tekrar teslimi ayırt edilmiyor	server.py:195
P2	Hatalı işi sıfırlama	Durum PENDING oluyor ama eski rc=7 ve hata bilgisi kalıyor; COALESCE, None ile temizlemiyor	server.py:309, db.py:281-284
P2 / doğrulama	Olmayan bağımlılığı ekleme	Hata yerine task_added; çözülemeyen bağımlılık saklanıyor	server.py:118-139, db.py:84-90
P2 / tutarlılık	Son görevi bitirip durum sorgulama	Tüm görevler bitse de oturum RUNNING görünüyor; ek get_next_action çağrısı gerekiyor	server.py:184-216, dag.py:170-178

İki istemci testi paralel hız/yük testi değildir: ardışık iki isteğin aynı görevi aldığını kanıtlar. Aynı süreçte yarış koşulu veya hesap havuzu davranışı ayrıca ölçülmedi. Sıfırlama testindeki ilk başarısız assert rc alanındadır; hata alanının da korunması kaynak kodundaki aynı COALESCE davranışıyla görülür.

Güçlü taraflar

Kurulumda JSON-RPC stdout akışı bozulmuyor; 8 araç keşfediliyor. Hatalı zorunlu argüman reddediliyor. Normal DAG sırası, döngü yakalama ve aşağı akışa SKIPPED yayılımı çalışıyor. Gerçek yerel komuttan alınan rc=7 başarılı sayılamadı. Yeni süreç aynı test veritabanını açınca yarım görevine devam etti. Bunlar faydalı bir temel; fiziksel güç kesintisi dayanıklılığı veya gerçek işçi kurtarması kanıtlanmış değil.

Somut sonraki adım

Önce record_result ve save_artifact sınırlarını düzeltmek gerekir: mevcut oturum/görev doğrulaması, yalnız atanmış görevden sonuç alma, sunucunun koştuğu kabul komutu, kök içinde çözülmüş dosya yolu. Sonra sahiplik/kira ve tekrar sonuç güvenliği eklenip iteration/reset tutarlılığı kapatılmalı. Bu 10 sınır testi yeşile dönmeden çoklu işçi kullanımına geçilmesini önermiyorum. Model/havuz entegrasyonu bunun ardından ayrı bir iş olarak ele alınmalı.

Bu incelemede sistem kodu ve bağlantı ayarları değiştirilmedi. Gemini/AGY/filo yolları çalıştırılmadı; model veya dış ağ çağrısı yapılmadı. Canlı orchestrator.db açılmadı. Testlerde ayrı geçici veritabanları ve çıktı dizinleri kullanıldı. Sunucunun mevcut log mekanizması test çağrılarını kendi loguna yazabilir. Yük, çoklu hesap, model çıktı kalitesi ve gerçek proje uygulama/merge davranışı test kapsamında değil.

Yeniden üretme ve kanıtlar

EnesOS kökünde PowerShell:

```
& 'C:\Users\Enes\AppData\Local\Python\pythoncore-3.14-64\python.exe' -X utf8 'work\mcp-orchestrator-audit-20261016\audit.py'
```

Mevcut sürümde beklenen sonuç: **rc=1, 22 test, 10 başarısızlık**. Script çıktıları aynı denetim klasörüne yazar; kaynak dosyalarını değiştirmez.

• Tam test çıktısı (test-output.txt)

• Yapılandırılmış sonuçlar ve kaynak SHA-256 özetleri (test-results.json)

• Gerçek MCP sınır çağrılarının gözlemleri (wire-observations.json)

• Tekrar çalıştırılabilir denetim kodu (audit.py)

Değerlendirme bu testte hash'i alınan yerel sürümle sınırlıdır. Kanıtlar: test-results.json, wire-observations.json, test-output.txt; satır atıfları yukarıdaki kaynak klasörüne göredir.